Web Development Foundations

Week 7 - "From Working - To Clean"

Day 2: "Function Refactor Iteration"

1

Reconnect

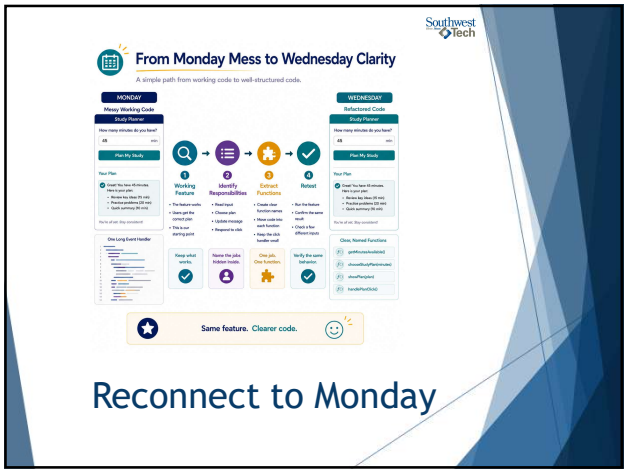
Monday:

- working code can still be hard to read
- long event handlers hide responsibilities
- functions name responsibilities
- refactoring must preserve behavior

Today:

- We perform the refactor one step at a time.

2



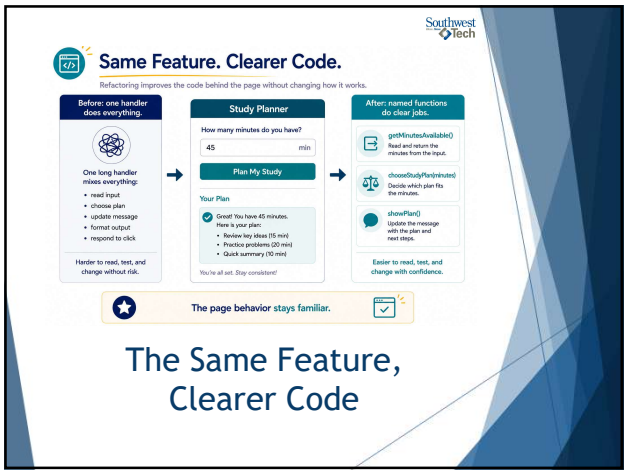
Reconnect to Monday

3

The Same Feature, Clearer Code

The feature:	*enter minutes and receive a study plan.*
The improvement:	*separate the jobs.*
Before:	*one event handler does everything.*
After:	*named functions each do one clear job.*

4



The Same Feature, Clearer Code

5

What Counts as Success Today

- the same CSS file styles more than one page
- navigation looks consistent
- text is readable
- spacing is less crowded
- one class-based style is used on purpose

6

Refactor In Small Steps

Do not refactor everything at once.

Use this rhythm:

1. choose one responsibility
2. move that code into a function
3. reconnect the call
4. retest
5. then choose the next responsibility

7

Small-Step Refactoring Rhythm

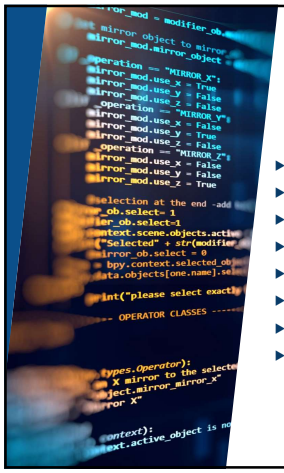
Make steady progress with small, testable steps.

1. choose one responsibility — Pick one job that is hidden inside the handler.
2. move it into a function — Create a function with a clear name for that job.
3. reconnect the call — Call the new function from the handler or another function.
4. retest — Run the feature. Confirm it still works the same.
5. choose the next responsibility — Repeat the cycle with the next responsibility.

Do not refactor everything at once.

Small-Step Refactor

8



What We Will Use Today

- expected behavior
- named function
- return value
- parameter
- event listener
- callback
- arrow function awareness
- retest

9

Today's Refactoring Toolbox

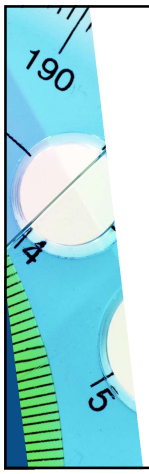
Tools for turning messy working code into clear, well-structured code.



Use the right tool. One step at a time.

Refactoring Toolbox

10



Before You Change Structure, Record Behavior

Before refactoring, record expected behavior.

Example test values:

- 0 minutes
- 10 minutes
- 30 minutes
- 60 minutes

After refactoring, run the same checks.

11

Record Behavior Before Refactoring

Capture what the feature does so you can confirm it stays the same.

Before Refactor — Test These Values			After Refactor — Retest the Same Values		
0	0 minutes	Shows short plan message	0	0 minutes	Shows short plan message
10	10 minutes	Shows short plan message	10	10 minutes	Shows short plan message
30	30 minutes	Shows medium plan message	30	30 minutes	Shows medium plan message
60	60 minutes	Shows long plan message	60	60 minutes	Shows long plan message
Behavior recorded and confirmed. This is our baseline.			Behavior matches the baseline. The feature still works the same.		

Use the same values and expected behavior after refactoring.

Retest the same values.

Record Behavior First

12

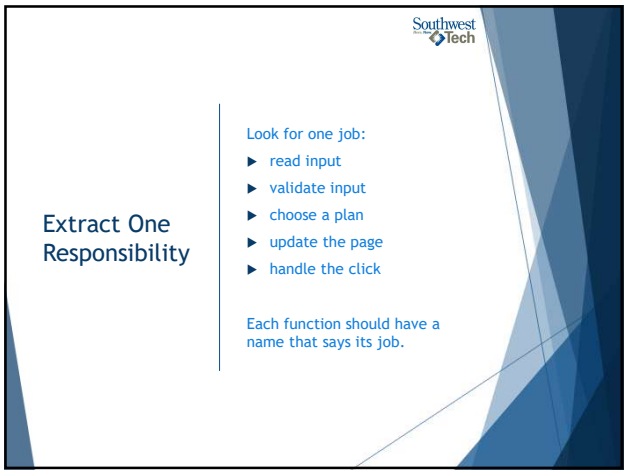
Extract One Responsibility

Look for one job:

- read input
- validate input
- choose a plan
- update the page
- handle the click

Each function should have a name that says its job.

13



Extract Responsibility

14

Return Values Carry Results Back

A return value sends a result back.

Example:

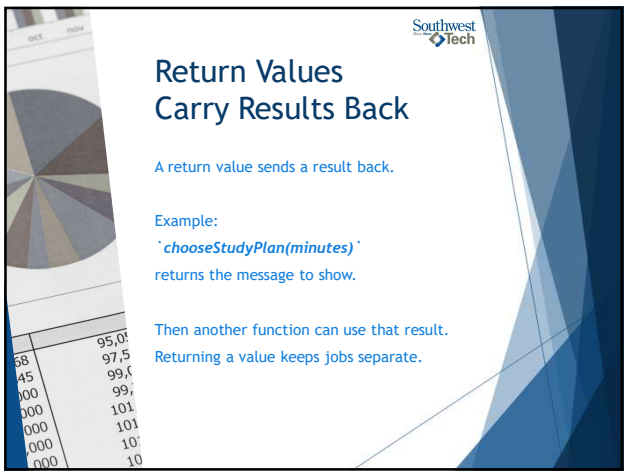
`` `chooseStudyPlan(minutes)` ``

returns the message to show.

Then another function can use that result.

Returning a value keeps jobs separate.

15



Return Value

16

Demo

"Refactor Into Named Functions"

17

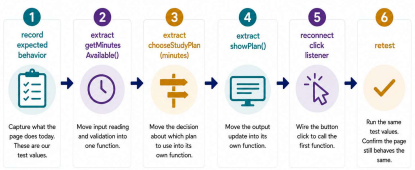
What to Watch For

- record expected behavior
- extract `` `getMinutesAvailable()` ``
- extract `` `chooseStudyPlan(minutes)` ``
- extract `` `showPlan()` ``
- reconnect the click listener
- retest the same values

18

Refactor Into Named Functions

A steady path from working code to clearer structure.



Structure changes. Behavior stays verified.

Demo:
Refactor

19

Named Function Callback

Before:

`button.addEventListener("click", function () { … });`

After:

`button.addEventListener("click", showPlan);`

The named function makes the event's purpose visible.

20

Anonymous Callback vs. Named Callback

Same event. Clearer intent.

Before: Anonymous Callback

```
addEventListener(
  "click",
  function () {
    … your code here …
  }
)
```

What does this do?
The code runs, but the purpose is not obvious.

After: Named Callback

```
addEventListener(
  "click",
  showPlan
)
```

What does this do?
The name shows the purpose: show the study plan.

The named function makes the event's purpose visible.

Named Function Callback

21

Arrow Functions Are Alternate Syntax

Arrow functions are another way to write functions.

Traditional:

`function showPlan() { … }`

Arrow style:

`const showPlan = () => { … };`

For this course:

- *recognize arrow functions.*
- *Use clear names first.*

This Photo by Unknown Author is licensed under CC BY-SA-NC

22

Two Ways You May See a Function

Same job. Different syntax styles.

Traditional Function Syntax

A style you have seen.

```
function showPlan() {
  // decide the plan
  …
}
```

Defines a function with a name. Use it. Call it. Read it.

Both styles can do the same job.

Arrow Function Syntax

An alternative style you may see.

```
const showPlan = () => {
  // decide the plan
  …
};
```

Also defines a function. Often used for short, clear jobs.

Recognize arrow functions.
Use clear names first.

Arrow Function Awareness

23

Why Arrow Functions Became Common

Advantages:

- shorter syntax for small functions
- common in modern examples
- useful for callbacks and array methods
- avoids some `this` confusion in advanced code

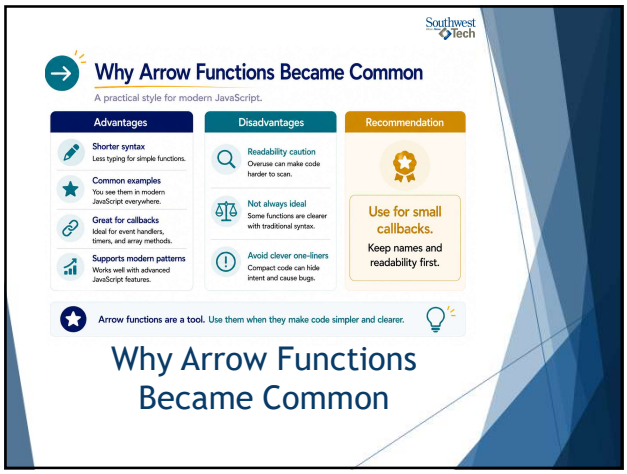
Disadvantages:

- can be harder to read at first
- not ideal for every named function
- easy to overuse for clever one-liners

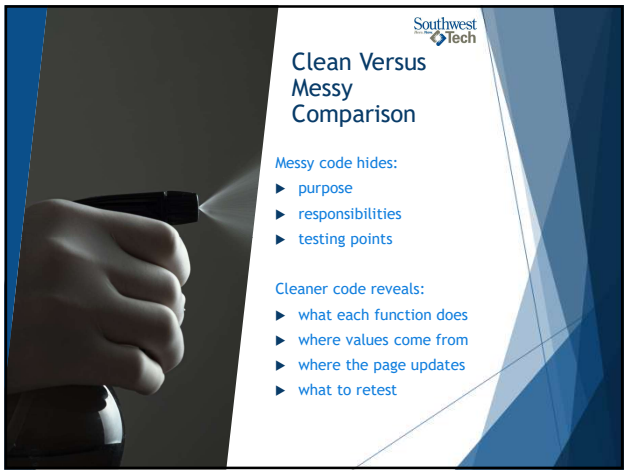
Recommendation:

- start using arrow functions for small callbacks,
- but keep names and readability first.

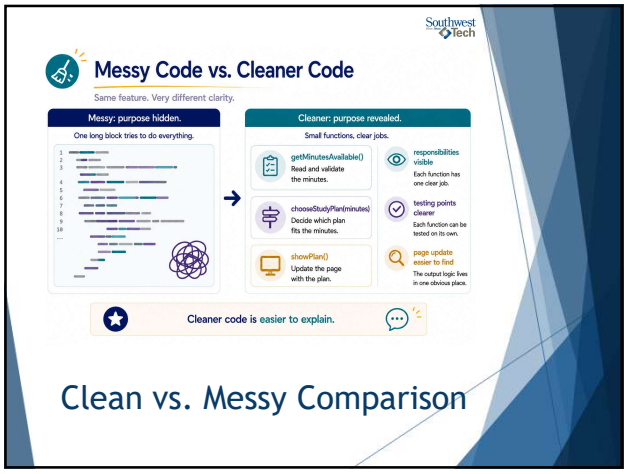
24



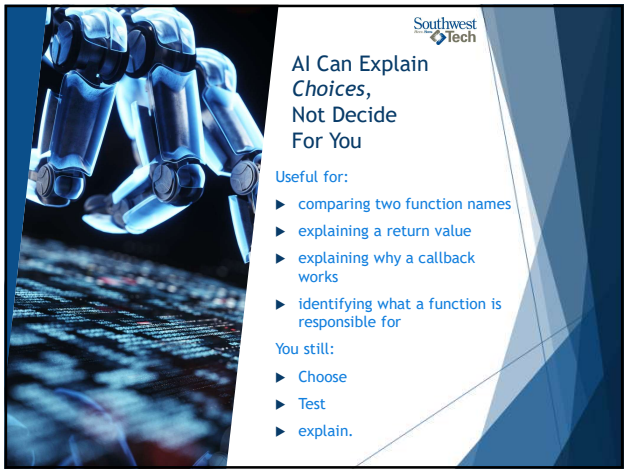
25



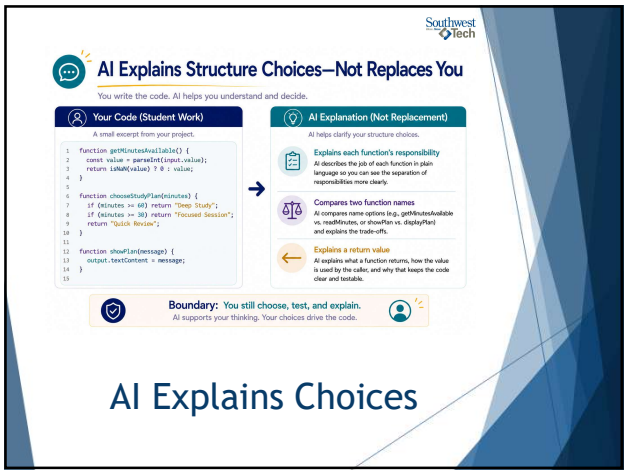
26



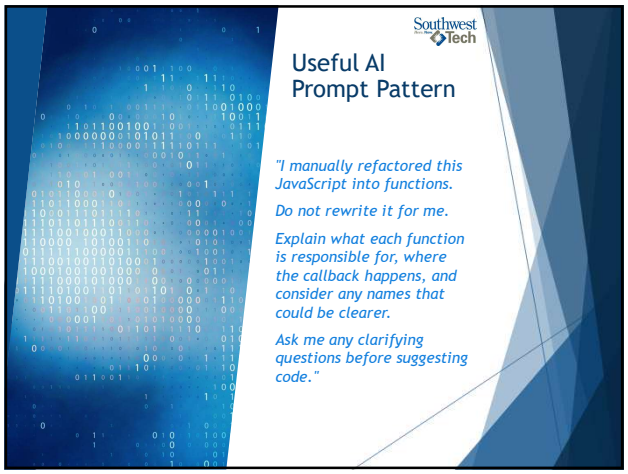
27



28



29



30

AI Prompt Pattern

For Explaining a JavaScript Refactor

Context	I manually refactored this JavaScript into functions.
Constraint	Do not rewrite it for me.
Explain	Please explain the code's: • responsibilities • callback • naming choices
Ask First	Ask me a question before suggesting code.

Prompt for explanation, not replacement.

Useful AI Prompt Pattern

31



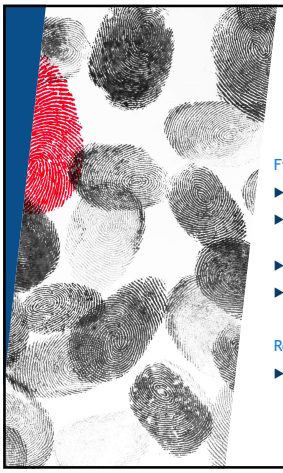
Lab Refinement

Refine structure

Your goal:

- improve function names
- organize logic clearly
- reduce repetition where possible
- keep the feature working
- add one clarity improvement

32



Evidence & Reflection

Final Assignment 7 evidence:

- updated HTML, CSS, and JS
- JavaScript uses functions effectively
- function names are meaningful
- behavior still works

Reflection

- Explain what changed

33

How To Read Next Week's Material

Required:

- read the async handout for the big idea
- focus on what happens now vs what happens later

Skim:

- Ajax and JSON as background
- JSON structure guide for objects and arrays

Reference:

- MDN links are there when you need exact syntax

34

Clear Structure Prepares You for Async Timing Next Week

1	2	3	4
Clear Functions	Verified Behavior	Timing Question	Async Next
Each function has one job. Responsibilities are visible.	You test the same values before and after refactoring. Behavior stays the same.	What happens if parts of the work take more time? What order do things complete?	Next week we'll explore how timing affects order and user experience.

Clear structure makes timing easier to follow.

Structure To Async Bridge

35



Closing Success Check

This week:

- working code became clearer
- responsibilities became functions
- callbacks became easier to read
- behavior stayed verified

Next:

- systems respond over time.

36



37